

Tarefa 15: Simulação da Difusão de Calor

Este documento apresenta a análise de desempenho de três abordagens de paralelização utilizando MPI para o problema de simulação de difusão de calor em uma malha 1D. O tamanho do array simulado (`bar_size`) foi de 10^8 posições, executado ao longo de 500 passos de tempo. As três abordagens requeridas foram:

1. Bloqueante simples (MPI_Send / MPI_Recv).
2. Não bloqueante com espera (MPI_Isend / MPI_Irecv e MPI_Wait).
3. Não bloqueante com sobreposição de computação (MPI_Isend / MPI_Irecv e MPI_Test).

1. Implementação sequencial

Como baseline, foi implementada uma versão sequencial. Ela se resume em:

1. Pela quantidade de passos
2. Aplique diferenças finitas nos vetores
3. Inverta os vetores

```
void simulate_seq(int steps, int bar_size) {  
    float *t1 = init(bar_size, 0);  
    float *t2 = init(bar_size, 0);  
  
    for (int j = 0; j < steps; j++) {  
        apply_one_step(t1, t2, bar_size);  
        swap(&t1, &t2);  
    }  
  
    free(t1);  
    free(t2);  
}
```

Por ser uma implementação puramente sequencial, não há ganho de desempenho ao aumentar o número de processos. Dessa forma, foi testado apenas com 1 processo.

1. 1. Utilização de dois vetores

A necessidade de utilizar dois vetores decorre da dependência de dados inerente ao método de diferenças finitas. Para calcular a temperatura de uma posição no passo de tempo atual, o algoritmo exige os valores daquela mesma posição e de seus vizinhos imediatos pertencentes ao passo de tempo anterior. Se a atualização fosse feita no próprio vetor, a modificação de uma célula corromperia o valor original que a célula vizinha precisaria utilizar no cálculo seguinte.

Para resolver esse problema de forma eficiente, adota-se a técnica de **double buffering** (duplo buffer): um vetor atua estritamente como leitura (estado anterior) e o outro como escrita (novo estado). Ao final de cada passo de tempo, a função `swap` apenas permuta os ponteiros desses vetores. Essa abordagem alterna a função de cada buffer com custo computacional constante $O(1)$, eliminando a necessidade de copiar massivamente os dados na memória para a próxima iteração.

2. Implementação Paralela

Para a paralelização, adotou-se a estratégia de **decomposição de domínio**. A barra 1D global é dividida em trechos menores, de modo que cada processo fica responsável por um bloco de tamanho `local_bar_size = bar_size / world_size`.

Como o cálculo de um ponto via diferenças finitas necessita dos valores adjacentes, os pontos localizados nas extremidades do bloco precisam de informações que pertencem aos processos vizinhos. Para suprir essa dependência, cada processo aloca o vetor com tamanho `local_bar_size + 2`.

Dessa forma, o ciclo principal de cada processo consiste em:

1. **Comunicação:** Trocar as bordas reais com as células fantasmas dos vizinhos (`rank - 1` e `rank + 1`).
2. **Computação:** Iterar pela porção do array sob sua responsabilidade e atualizar os estados.
3. **Condição de Contorno:** O processo raiz (`rank 0`) impõe uma temperatura constante (ex: 100.0) no início da barra para simular uma fonte de calor contínua.

```
void simulate_send_recv(int steps, int local_bar_size, int rank, int world_size) {
    float *t1 = init(local_bar_size + 2, rank);
    float *t2 = init(local_bar_size + 2, rank);

    // Vizinho esquerdo
    int v_esq = rank == 0 ? MPI_PROC_NULL : rank - 1;
    // Vizinho direito
    int v_dir = rank == world_size - 1 ? MPI_PROC_NULL : rank + 1;

    // Sincroniza todos os processos
    MPI_Barrier(MPI_COMM_WORLD);

    for (int step = 0; step < steps; step++) {
        // Comunica a ponta de cada processo
        communicate(rank, v_esq, v_dir, t1, local_bar_size);

        // Processa 1 step
        apply_one_step(t1, t2, local_bar_size);

        swap(&t1, &t2);

        if (rank == 0) {
            t1[0] = 100.0;
        }
    }

    free(t1);
    free(t2);
}
```

2. 1. Implementação bloqueante

A implementação utilizando funções bloqueantes (`MPI_Send` e `MPI_Recv`) exige um controle rigoroso da ordem de comunicação. Como essas funções interrompem a execução do processo até que a transferência de dados na rede seja validada, uma abordagem ingênua — onde todos tentassem enviar dados simultaneamente antes de chamar a função de receber — causaria um deadlock. Nesse cenário, o programa travaria por completo, pois todos os processos ficariam aguardando indefinidamente por um receptor que também estaria bloqueado tentando enviar.

Para garantir que o fluxo de dados ocorra de maneira segura, a solução adotada separa os processos pares e ímpares. A comunicação é dividida em duas lógicas distintas, baseadas na paridade do identificador (rank) de cada processo:

- **Processos Pares:** Iniciam o ciclo executando as operações de envio (MPI_Send) de suas respectivas bordas. Apenas após a conclusão dessa etapa, eles realizam as chamadas de recebimento (MPI_Recv) para obter os dados dos vizinhos.
- **Processos Ímpares:** Executam a operação complementar. Primeiro, eles aguardam ativamente nas funções de recebimento (MPI_Recv) pelas mensagens de seus vizinhos e, somente após os dados chegarem, realizam o envio (MPI_Send) de suas próprias bordas.

Essa estratégia garante o casamento perfeito das operações em ambas as direções (esquerda e direita). Sempre que um processo par está tentando enviar seus dados, existe a garantia estrutural de que o seu vizinho ímpar já está pronto na rotina de recebimento aguardando por essa mesma mensagem, o que elimina qualquer risco de deadlock e mantém o fluxo do algoritmo eficiente.

```
void communicate(int rank, int esq, int dir, float *bar, int bar_size) {
    // Os pares enviam primeiro e recebem depois
    if (rank % 2 == 0) {
        if (dir != MPI_PROC_NULL) {
            // Envia o último que consigo processar
            MPI_Send(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD);
            // Receber o que é processado pelo "vizinho da direita"
            MPI_Recv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
        }

        if (esq != MPI_PROC_NULL) {
            // Envia o primeiro que consegue processar
            MPI_Send(&bar[1], 1, MPI_FLOAT, esq, 2, MPI_COMM_WORLD);
            // Recebe o que é processado pelo "vizinho da esquerda"
            MPI_Recv(&bar[0], 1, MPI_FLOAT, esq, 3, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
        }
    }
    // Os ímpares recebem primeiro e enviam depois.
    else {
        if (esq != MPI_PROC_NULL) {
            // Recebe o que é processado pelo "vizinho da esquerda"
            MPI_Recv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            // Envia o primeiro que consegue processar
            MPI_Send(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD);
        }

        if (dir != MPI_PROC_NULL) {
            // Receber o que é processado pelo "vizinho da direita"
            MPI_Recv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 2, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            // Envia o último que consigo processar
            MPI_Send(&bar[bar_size], 1, MPI_FLOAT, dir, 3, MPI_COMM_WORLD);
        }
    }
}
```

2. 2. Implementação não-bloqueante

A abordagem não-bloqueante simplifica drasticamente o código ao eliminar a necessidade da estratégia de coloração par-ímpar. Com o uso de `MPI_Isend` e `MPI_Irecv` (onde o 'I' indica Immediate), a ocorrência de deadlocks estruturais torna-se praticamente impossível.

Ao invés de pausar o programa até que a mensagem seja completamente transmitida pela rede, essas funções apenas “postam” (agendam) a intenção de comunicar e retornam o controle ao processo atual instantaneamente. Devido a essa natureza assíncrona, todos os processos podem iniciar simultaneamente suas operações de recebimento e, logo na sequência, iniciar seus envios, sem o risco de ficarem bloqueados esperando uns pelos outros.

A estrutura `MPI_Request` atua como um rastreador (“handle”) para cada transferência iniciada. O código incrementa o contador `req_count` para cada operação válida (ignorando os nós inexistentes nas bordas da malha global, representados por `MPI_PROC_NULL`).

O ponto crucial dessa implementação está na linha final: o `MPI_Waitall`. Essa função atua como uma barreira de sincronização local obrigatória. Ela garante que, embora as operações tenham sido iniciadas em segundo plano de forma não-bloqueante, o programa atual não avançará para aplicar o método numérico (cálculo das diferenças finitas) até que todas as requisições atreladas ao vetor `reqs` tenham sido concluídas com sucesso, assegurando que as células fantasmas contenham os dados corretos e atualizados.

```
void communicate(int rank, int esq, int dir, float *bar, int bar_size) {
    // Criamos um vetor para rastrear até 4 requisições (2 envios + 2
    // recebimentos)
    MPI_Request reqs[4];
    int req_count = 0;

    // Postar os recebimentos
    if (esq != MPI_PROC_NULL) {
        MPI_Irecv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }
    if (dir != MPI_PROC_NULL) {
        MPI_Irecv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }

    // Postar os envios
    if (esq != MPI_PROC_NULL) {
        MPI_Isend(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }
    if (dir != MPI_PROC_NULL) {
        MPI_Isend(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }

    // Sincroniza
    MPI_Waitall(req_count, reqs, MPI_STATUSES_IGNORE);
}
```

2. 3. Implementação não-bloqueante com test

Esta versão implementa o conceito de sobreposição (overlap) entre comunicação e computação, um dos grandes objetivos da programação paralela de alto desempenho. A ideia central é manter a CPU trabalhando ativamente enquanto a placa de rede transfere os dados em segundo plano.

Para que isso seja possível, a lógica do algoritmo precisou ser refatorada para dividir o domínio em duas partes distintas:

- **Pontos Internos (índices 2 a `local_bar_size - 1`):** Podem ser calculados imediatamente, pois o cálculo das diferenças finitas para eles depende apenas dos dados que o próprio processo já possui.
- **Pontos de Borda (índices 1 e `local_bar_size`):** São estritamente dependentes das células fantasmas (0 e `local_bar_size + 1`) que estão sendo recebidas via rede.

Nesta implementação, a função `communicate` apenas posta as mensagens na rede e devolve o controle instantaneamente. Em seguida, o processo inicia o cálculo dos pontos internos.

Para equilibrar o uso da CPU entre o cálculo numérico e o monitoramento da rede, adotou-se o processamento em blocos (chunks) de 64 pontos. Ao final de cada bloco calculado, o código invoca `MPI_Testall`. Diferente do `MPI_Waitall`, o `Testall` não bloqueia a execução; ele apenas verifica o status (polling) das requisições assíncronas. Se os dados tiverem chegado, a flag `comm_concluida` é marcada.

Caso o processo termine de calcular todos os pontos internos e a comunicação ainda não tenha finalizado, o `MPI_Waitall` atua como um fallback (garantia), bloqueando o processo apenas pelo tempo restante necessário. Por fim, com a garantia de que as células fantasmas chegaram, o programa calcula os pontos de borda, fechando o ciclo.

Como visto na análise de resultados anterior, essa técnica exige um ajuste fino: caso a quantidade de computação local (volume de pontos internos) seja muito grande e a latência da rede muito baixa, o tempo gasto checando repetidamente o `MPI_Testall` acaba gerando um overhead que pode ser superior ao ganho da sobreposição, afetando o desempenho (como ocorreu no cenário de apenas 2 processos).

```
void communicate(int rank, int esq, int dir, float *bar, int bar_size,
                 MPI_Request *reqs) {
    int req_count = 0;

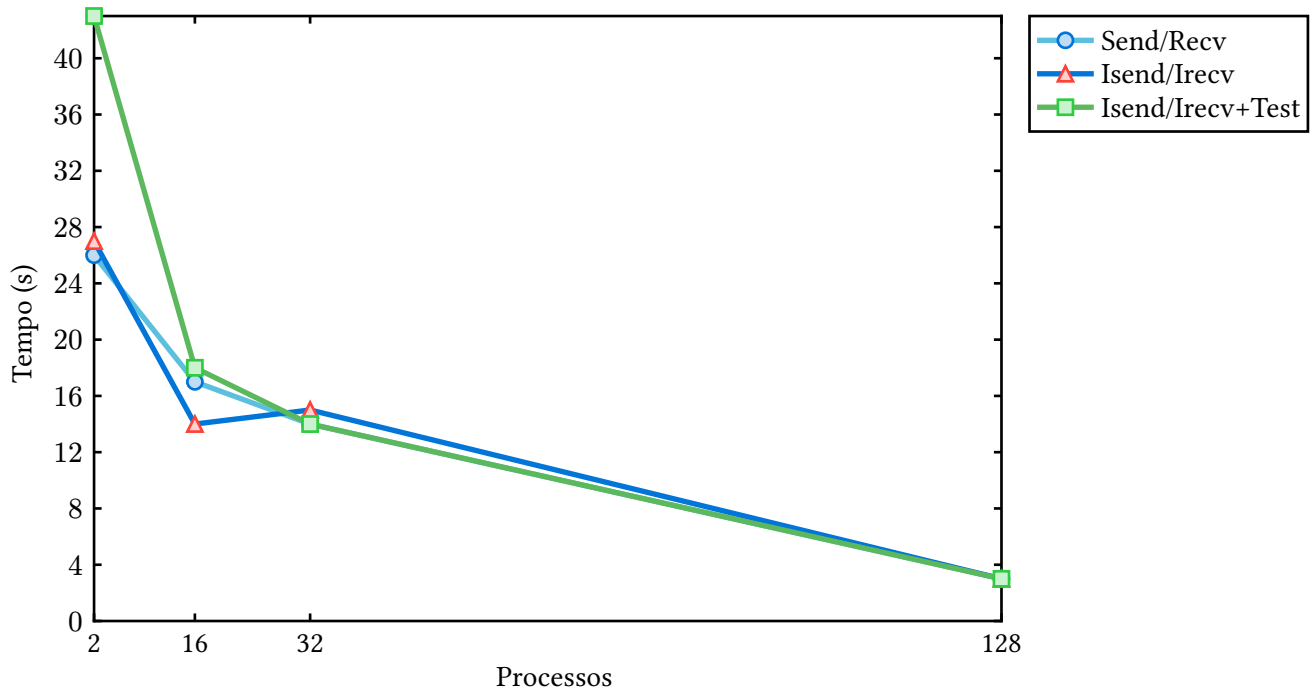
    if (esq != MPI_PROC_NULL) {
        MPI_Irecv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
                  &reqs[req_count++]);
        MPI_Isend(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD,
                  &reqs[req_count++]);
    }

    if (dir != MPI_PROC_NULL) {
        MPI_Irecv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
                  &reqs[req_count++]);
        MPI_Isend(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD,
                  &reqs[req_count++]);
    }

    return req_count;
}
```

3. Resultados Coletados

O gráfico abaixo organiza o tempo de execução (em segundos) variando o número de processos (threads MPI) de 1 a 512, conforme dados obtidos na experimentação.



4. Anexo

4. 1. Versão sequencial

```
#include "mpi.h"
#include "util.c"
#include <stdio.h>
#include <stdlib.h>

void apply_one_step(float *t1, float *t2, int size) {
    for (int i = 1; i < size - 1; i++) {
        t2[i] = atualizar_ponto(t1, i);
    }
    t2[0] = t2[1];
    t2[size - 1] = t2[size - 2];
}

double simulate_seq(int steps, int bar_size) {
    // Inicialização também não entra no tempo de execução.
    float *t1 = init(bar_size, 0);
    float *t2 = init(bar_size, 0);

    // Como vou compilar com o mpicc de qualquer modo, aproveito pra usar a função
    // dele.
    double start = MPI_Wtime();

    for (int j = 0; j < steps; j++) {
        apply_one_step(t1, t2, bar_size);
        swap(&t1, &t2);
    }
}
```

```

}

double elapsed = MPI_Wtime() - start;

// Liberação de memória não entra no tempo de execução.
free(t1);
free(t2);

return elapsed;
}

int main() {
    int sizes[6] = {1e2, 1e4, 1e5, 1e6, 1e7, 1e8};

    int steps = 500;

    for (int i = 0; i < 6; i++) {
        int size = sizes[i];

        double elapsed = simulate_seq(steps, size);

        printf(
            "Processamento finalizado de %d iterações... tempo de execução: %f\n",
            size, elapsed);
    }
}

```

4. 2. Versão com send e recv

```

#include "mpi.h"
#include "util.c"
#include <stdio.h>
#include <stdlib.h>

void apply_one_step(float *t1, float *t2, int size) {
    for (int i = 1; i <= size; i++) {
        t2[i] = atualizar_ponto(t1, i);
    }
}

void communicate(int rank, int esq, int dir, float *bar, int bar_size) {
    // Os pares enviam primeiro e recebem depois
    if (rank % 2 == 0) {
        if (dir != MPI_PROC_NULL) {
            // Envia o último que consigo processar
            MPI_Send(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD);
            // Receber o que é processado pelo "vizinho da direita"
            MPI_Recv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        }

        if (esq != MPI_PROC_NULL) {
            // Envia o primeiro que consegue processar
            MPI_Send(&bar[1], 1, MPI_FLOAT, esq, 2, MPI_COMM_WORLD);

```

```

    // Recebe o que é processado pelo "vizinho da esquerda"
    MPI_Recv(&bar[0], 1, MPI_FLOAT, esq, 3, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
}
}
// Os impares recebem primeiro e enviam depois.
else {
    if (esq != MPI_PROC_NULL) {
        // Recebe o que é processado pelo "vizinho da esquerda"
        MPI_Recv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        // Envia o primeiro que consegue processar
        MPI_Send(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD);
    }

    if (dir != MPI_PROC_NULL) {
        // Receber o que é processado pelo "vizinho da direita"
        MPI_Recv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 2, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        // Envia o último que consigo processar
        MPI_Send(&bar[bar_size], 1, MPI_FLOAT, dir, 3, MPI_COMM_WORLD);
    }
}
}

double simulate_send_recv(int steps, int local_bar_size, int rank,
                          int world_size) {
    // Inicialização também não entra no tempo de execução.
    float *t1 = init(local_bar_size + 2, rank);
    float *t2 = init(local_bar_size + 2, rank);

    // Vizinho esquerdo
    int v_esq = rank == 0 ? MPI_PROC_NULL : rank - 1;
    // Vizinho direito
    int v_dir = rank == world_size - 1 ? MPI_PROC_NULL : rank + 1;

    MPI_Barrier(MPI_COMM_WORLD);

    printf("Iniciando processamento de %d passos no processo %d .\n", steps,
           rank);

    // Como vou compilar com o mpicc de qualquer modo, aproveito pra usar a função
    // dele.
    double start = MPI_Wtime();

    for (int step = 0; step < steps; step++) {
        // Comunica a ponta de cada processo
        communicate(rank, v_esq, v_dir, t1, local_bar_size);

        // Processa 1 step
        apply_one_step(t1, t2, local_bar_size);

        swap(&t1, &t2);

        if (rank == 0) {

```



```

        t1[0] = 100.0;
    }
}
double elapsed = MPI_Wtime() - start;

// Liberação de memória não entra no tempo de execução.
free(t1);
free(t2);

return elapsed;
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Utilize pelo menos 2 processos MPI para rodar esse programa.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int bar_size = 1e8;
    int bar_per_process = bar_size / size;

    int steps = 500;

    if (rank == 0) {
        printf("Simulando difusao de calor na barra 1D...\n");
        printf("Tamanho global: %d pontos | Processos: %d | Passos: %d\n", bar_size,
            size, steps);
        printf("-----\n");
    }

    double elapsed = simulate_send_recv(steps, bar_per_process, rank, size);

    if (rank == 0) {
        printf(
            "Processamento finalizado de %d iterações... tempo de execução: %f\n",
            steps, elapsed);
    }

    MPI_Finalize();

    return 0;
}

```

4. 3. Versão com Isend e Irecv

```

#include "mpi.h"
#include "util.c"
#include <stdio.h>
#include <stdlib.h>

void apply_one_step(float *t1, float *t2, int size) {
    for (int i = 1; i <= size; i++) {
        t2[i] = atualizar_ponto(t1, i);
    }
}

void communicate(int rank, int esq, int dir, float *bar, int bar_size) {
    // Criamos um vetor para rastrear até 4 requisições (2 envios + 2
    // recebimentos)
    MPI_Request reqs[4];
    int req_count = 0;

    // Postar os recebimentos
    if (esq != MPI_PROC_NULL) {
        MPI_Irecv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }
    if (dir != MPI_PROC_NULL) {
        MPI_Irecv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }

    // Postar os envios
    if (esq != MPI_PROC_NULL) {
        MPI_Isend(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }
    if (dir != MPI_PROC_NULL) {
        MPI_Isend(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD,
            &reqs[req_count++]);
    }

    // Sincroniza
    MPI_Waitall(req_count, reqs, MPI_STATUSES_IGNORE);
}

double simulate_isend_irecv(int steps, int local_bar_size, int rank,
    int world_size) {
    // Inicialização também não entra no tempo de execução.
    float *t1 = init(local_bar_size + 2, rank);
    float *t2 = init(local_bar_size + 2, rank);

    // Vizinho esquerdo
    int v_esq = rank == 0 ? MPI_PROC_NULL : rank - 1;
    // Vizinho direito
    int v_dir = rank == world_size - 1 ? MPI_PROC_NULL : rank + 1;

    MPI_Barrier(MPI_COMM_WORLD);

```

```

printf("Iniciando processamento de %d passos no processo %d .\n", steps,
      rank);

// Como vou compilar com o mpicc de qualquer modo, aproveito pra usar a função
// dele.
double start = MPI_Wtime();

for (int step = 0; step < steps; step++) {
    // Comunica a ponta de cada processo
    communicate(rank, v_esq, v_dir, t1, local_bar_size);

    // Processa 1 step
    apply_one_step(t1, t2, local_bar_size);

    swap(&t1, &t2);

    if (rank == 0) {
        t1[0] = 100.0;
    }
}
double elapsed = MPI_Wtime() - start;

// Liberação de memória não entra no tempo de execução.
free(t1);
free(t2);

return elapsed;
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Utilize pelo menos 2 processos MPI para rodar esse programa.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int bar_size = 1e8;
    int bar_per_process = bar_size / size;

    int steps = 500;

    if (rank == 0) {
        printf("Simulando difusao de calor na barra 1D...\n");
        printf("Tamanho global: %d pontos | Processos: %d | Passos: %d\n", bar_size,
              size, steps);
        printf("-----\n");
    }
}

```

```

double elapsed = simulate_isend_irecv(steps, bar_per_process, rank, size);

if (rank == 0) {
    printf(
        "Processamento finalizado de %d iterações... tempo de execução: %f\n",
        steps, elapsed);
}

MPI_Finalize();

return 0;
}

```

4. 4. Versão com Isend e Irecv + Test

```

#include "mpi.h"
#include "util.c"
#include <stdio.h>
#include <stdlib.h>

void apply_one_step(float *t1, float *t2, int size) {
    for (int i = 1; i <= size; i++) {
        t2[i] = atualizar_ponto(t1, i);
    }
}

int communicate(int rank, int esq, int dir, float *bar, int bar_size,
                MPI_Request *reqs) {
    int req_count = 0;

    if (esq != MPI_PROC_NULL) {
        MPI_Irecv(&bar[0], 1, MPI_FLOAT, esq, 0, MPI_COMM_WORLD,
                 &reqs[req_count++]);
        MPI_Isend(&bar[1], 1, MPI_FLOAT, esq, 1, MPI_COMM_WORLD,
                 &reqs[req_count++]);
    }

    if (dir != MPI_PROC_NULL) {
        MPI_Irecv(&bar[bar_size + 1], 1, MPI_FLOAT, dir, 1, MPI_COMM_WORLD,
                 &reqs[req_count++]);
        MPI_Isend(&bar[bar_size], 1, MPI_FLOAT, dir, 0, MPI_COMM_WORLD,
                 &reqs[req_count++]);
    }

    return req_count;
}

double simulate_isend_irecv(int steps, int local_bar_size, int rank,
                           int world_size) {
    // Inicialização também não entra no tempo de execução.
    float *t1 = init(local_bar_size + 2, rank);
    float *t2 = init(local_bar_size + 2, rank);
}

```

```

// Vizinho esquerdo
int v_esq = rank == 0 ? MPI_PROC_NULL : rank - 1;
// Vizinho direito
int v_dir = rank == world_size - 1 ? MPI_PROC_NULL : rank + 1;

MPI_Barrier(MPI_COMM_WORLD);

printf("Iniciando processamento de %d passos no processo %d .\n", steps,
      rank);

// Como vou compilar com o mpicc de qualquer modo, aproveito pra usar a função
// dele.
double start = MPI_Wtime();
for (int step = 0; step < steps; step++) {
    MPI_Request reqs[4];

    int req_count = communicate(rank, v_esq, v_dir, t1, local_bar_size, reqs);

    int comm_concluida = 0;
    int i = 2;

    while (i <= local_bar_size - 1) {
        for (int bloco = 0; bloco < 64 && i <= local_bar_size - 1; bloco++, i++) {
            t2[i] = atualizar_ponto(t1, i);
        }

        if (!comm_concluida) {
            MPI_Testall(req_count, reqs, &comm_concluida, MPI_STATUSES_IGNORE);
        }
    }

    if (!comm_concluida) {
        MPI_Waitall(req_count, reqs, MPI_STATUSES_IGNORE);
    }

    t2[1] = atualizar_ponto(t1, 1);

    if (local_bar_size > 1) {
        t2[local_bar_size] = atualizar_ponto(t1, local_bar_size);
    }

    swap(&t1, &t2);

    if (rank == 0) {
        t1[0] = 100.0;
    }
}

double elapsed = MPI_Wtime() - start;

// Liberação de memória não entra no tempo de execução.
free(t1);
free(t2);

return elapsed;

```

```

}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Utilize pelo menos 2 processos MPI para rodar esse programa.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int bar_size = 1e8;
    int bar_per_process = bar_size / size;

    int steps = 500;

    if (rank == 0) {
        printf("Simulando difusao de calor na barra 1D...\n");
        printf("Tamanho global: %d pontos | Processos: %d | Passos: %d\n", bar_size,
            size, steps);
        printf("-----\n");
    }

    double elapsed = simulate_isend_irecv(steps, bar_per_process, rank, size);

    if (rank == 0) {
        printf(
            "Processamento finalizado de %d iterações... tempo de execução: %f\n",
            steps, elapsed);
    }

    MPI_Finalize();

    return 0;
}

```

4. 5. Funções utilitárias

```

#include <stdio.h>
#include <stdlib.h>

float atualizar_ponto(float *arr, int i) {
    return arr[i] + (0.4 * (arr[i - 1] - (2 * arr[i]) + arr[i + 1]));
}

void print_arr(float *arr, int size) {
    if (size == 0) {
        printf("[]\n");
    }
}

```

```
printf("[");

for (int i = 0; i < size - 1; i++) {
    printf("%f, ", arr[i]);
}

printf("%f]\n", arr[size - 1]);
}

float *init(int size, int rank) {
    float *arr = (float *)malloc(size * sizeof(float));

    for (int i = 0; i < size; i++) {
        arr[i] = 0.0;
    }

    if (rank == 0) {
        arr[0] = 100.0;
    }

    return arr;
}

void swap(float **t1, float **t2) {
    float *TEMP = *t1;
    *t1 = *t2;
    *t2 = TEMP;
}
```